Modul 2.1 Oracle

1.3 Oracle

1.3.1 Pengenalan

Oracle *Database* adalah suatu perangkat lunak *database* yang digunakan untuk menyimpan data. Biasanya perusahaan besar yang menggunakan program ini. Aplikasi ini merupakan produk yang dikembangkan oleh Oracle Corporation, pertama kali dirilis pada tahun 1979. Oracle dikembangkan oleh Larry-Ellison bersama Bob Miner, dan Ed Oates yang pada awalnya bekerja di *Software Development Laboratories* (SDL) pada tahun 1977. SDL kemudian berhasil membuat versi *prototype* orisinil dari Oracle Software, yaitu Oracle versi 1, yang ditulis dalam bahasa C dan mempunyai SQL *interface*. Nama Oracle berasal dari *code-name* suatu proyek milik CIA dimana Larry-Ellison dulunya bekerja.

1.3.2 Fungsionalitas

Oracle *Relational Database Management System* (Oracle RDBMS) atau sederhananya Oracle, merupakan sebuah RDBMS yang mempunyai fungsi antara lain sebagai berikut :

1. *Data Storage* pada area perancangan basis data.
2. *Access Data* untuk aplikasi tertentu, dengan Teknik Optimisasi.
3. *Database Security* dan pengaturan *User Privilege* tertentu.
4. *Consistency Data* yang berfungsi untuk proteksi data, dan *locking-mechanism.*
5. *Integrity Data* yang berfungsi pada implementasi basis data tersebar.

1.3.3 Kelebihan

1. *Scalability*, kemampuan menangani banyak *user* yang melakukan koneksi secara simultan tanpa berkurangnya *performance* secara signifikan. Dalam dokumentasinya, Oracle menyebutkan bahwa database Oracle dapat melayani puluhan ribu *user* secara simultan.
2. *Reliability* yang bagus, yaitu kemampuan untuk melindungi data dari kerusakan jika terjadi kegagalan fungsi pada sistem seperti *disk failure*.
3. *Stability*, yaitu kemampuan untuk tidak *crash* karena beban yang tinggi.
4. *Availability*, yaitu kemampuan dalam penanganan *crash* atau *failure* agar *service* tetap berjalan normal.
5. *Multiplatform*, dapat digunakan pada banyak sistem operasi seperti Windows, Unix, Linux dan Solaris.
6. Mendukung data yang berukuran besar. Berdasarkan dokumentasinya, Oracle dapat menampung data sampai 512 petabyte (1 petabyte= 1024 terabyte).
7. *Security* yang cukup handal.

1.3.4 Kekurangan

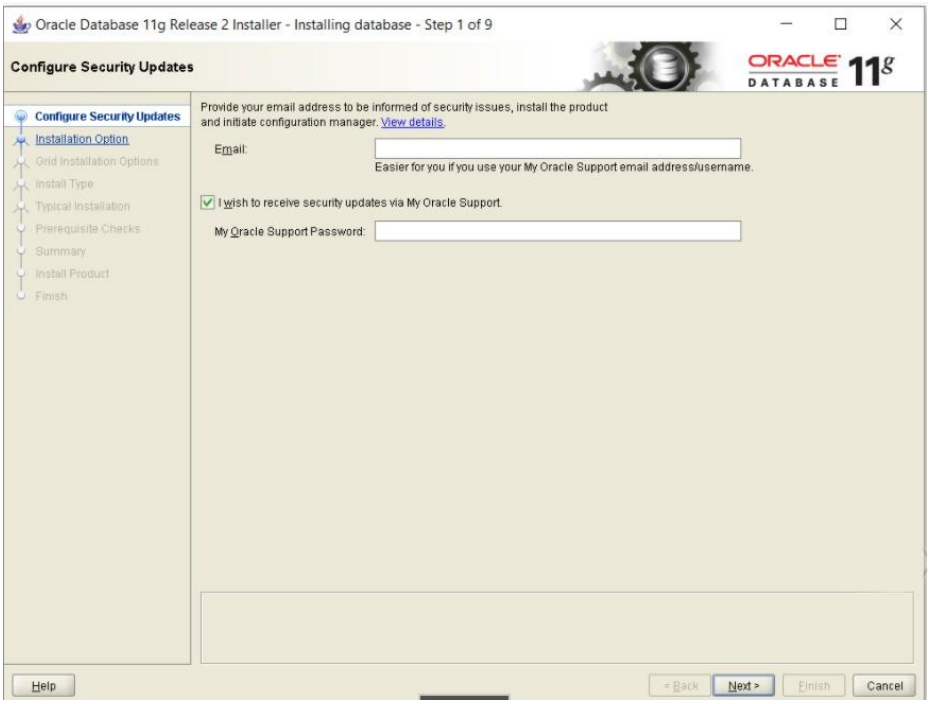
1. Merupakan *software* DMBS yang paling mahal, paling rumit, dan paling sulit untuk dipelajari.
2. Membutuhkan spesifikasi *hardware* yang tinggi untuk dapat menjalankan *software* DMBS Oracle supaya berjalan dengan stabil.
3. Hanya diperuntukan bagi perusahaan berukuran besar, dan tidak cocok untuk perusahaan kecil maupun menengah.
4. Data yang bertambah ukurannya akan mengalami kelambatan proses, jadi harus ada *database management.*
5. Harga yang sangat mahal untuk sebuah *database* dan penggunaan Oracle sangat memakan banyak biaya, mulai dari *device* sampai diperlukannya DBA yang handal.

1.3.5 Instalasi Oracle

1. Double click pada setup oracle yang telah diunduh setup.exe
2. Setelah itu click yes pada oracle installation

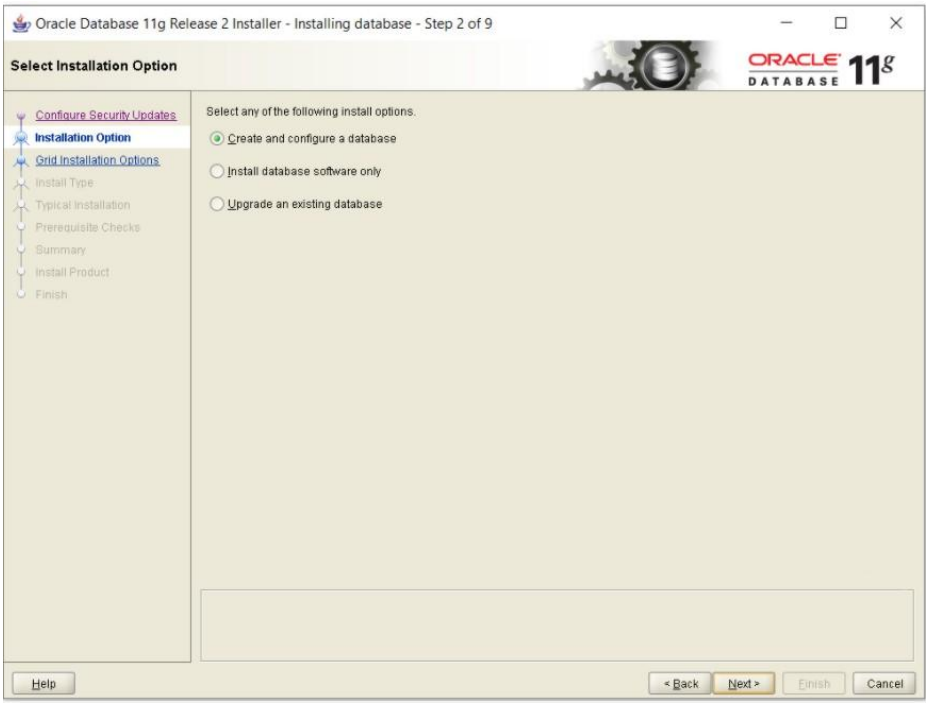
Gambar 1-18. Oracle *installation*

3. Masukkan email untuk mendapatkan update dari oracle (opsional)



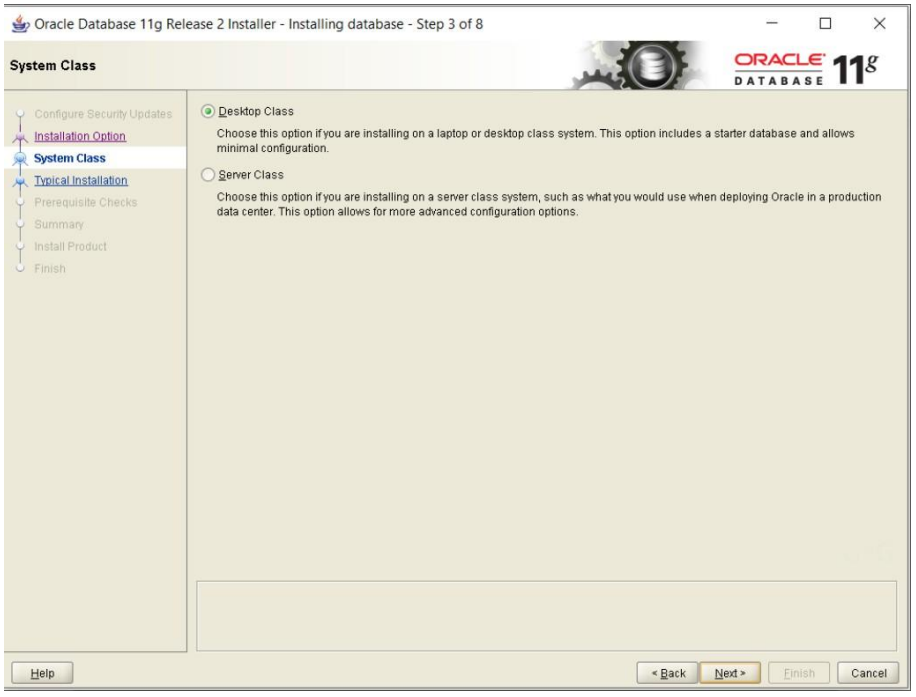
Gambar 1-19. Configure security updates

4. Create and configure a database



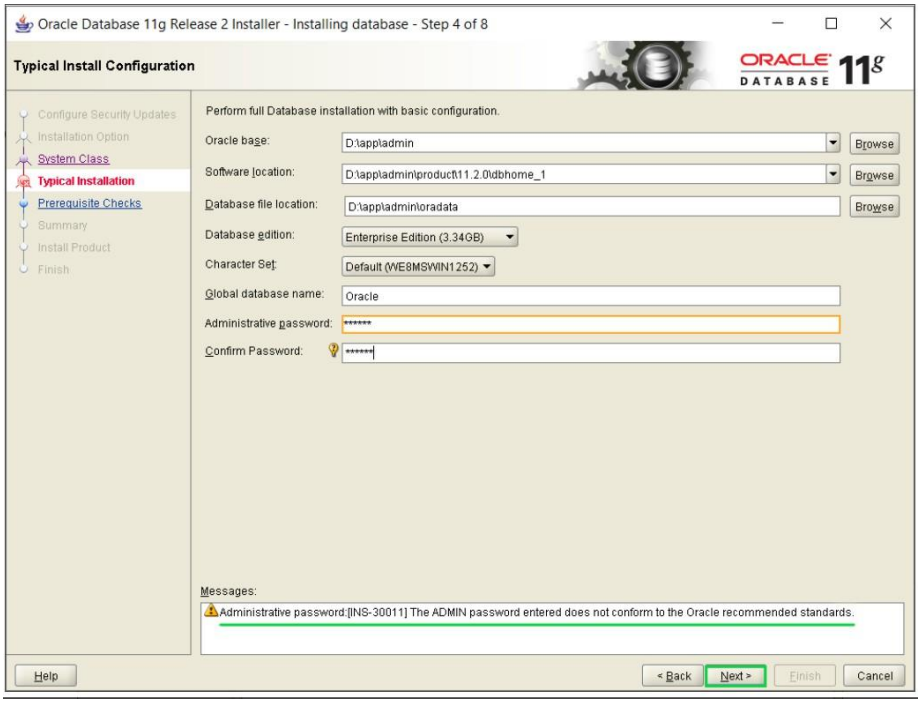
Gambar 1-20. Oracle installation option

5. Pilih dekstop class



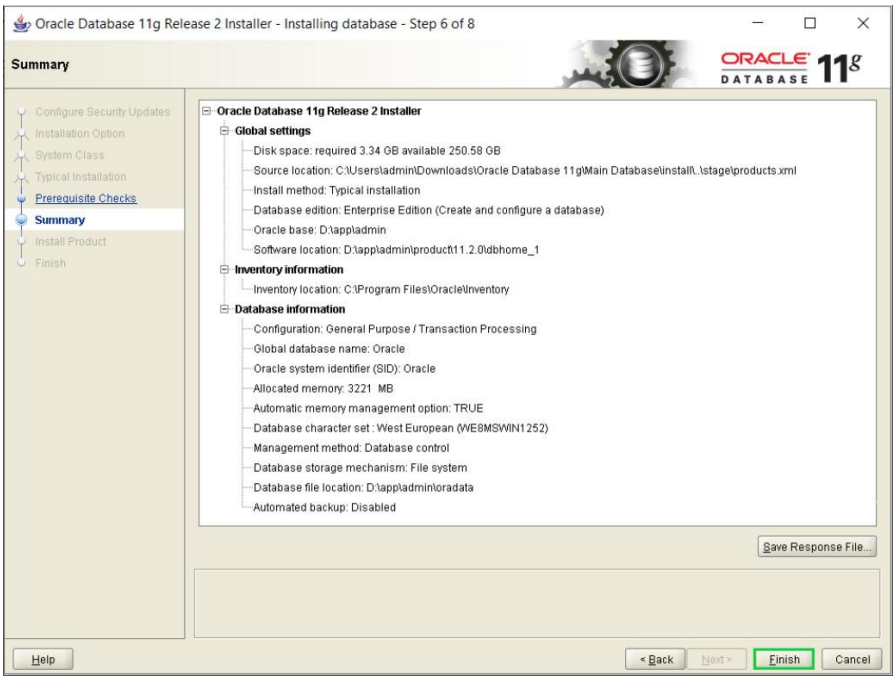
Gambar 1-21. Oracle installation system class

6. Konfigurasi sesuai kebutuhan user



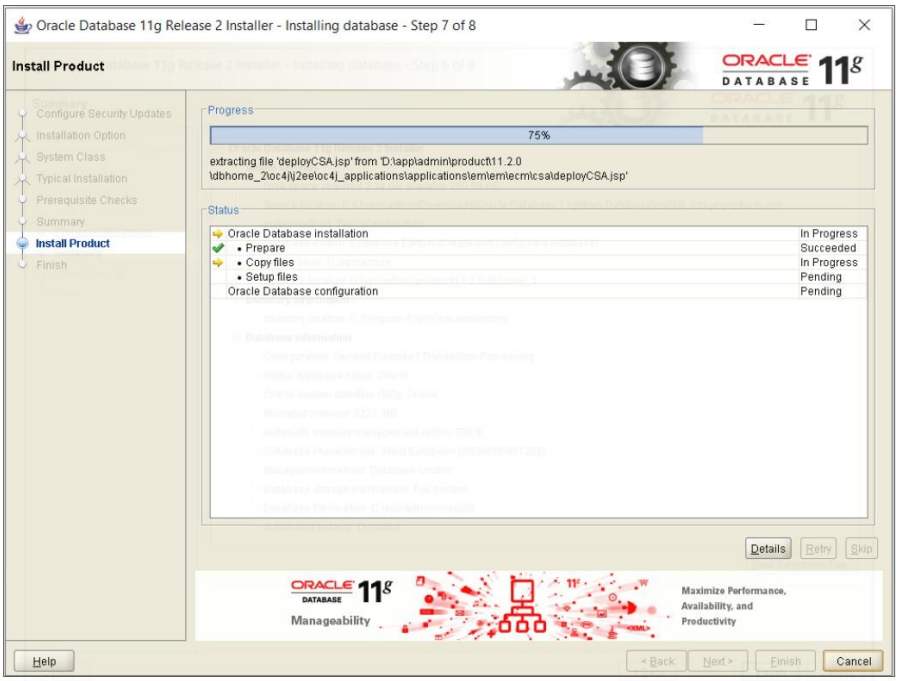
Gambar 1-22. Oracle typical installation

7. Pengecekan hardware oleh oracle



Gambar 1-23. Oracle installation summary

8. Oracle installation



Gambar 1-24. Oracle installation

9. Click ok untuk menyelesaikan installasi

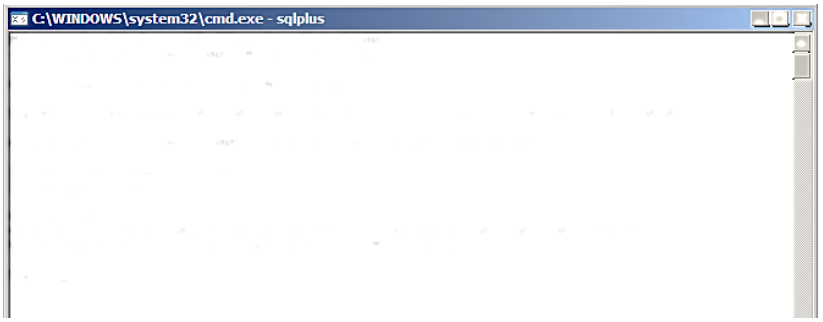
Gambar 1-25 Oracle installation

1.3.6 Cara menjalankan Oracle 11g

Ada 3 cara menjalankan Oracle pada PC,yaitu :

1. ***Command prompt***

   Untuk membuka *command prompt,* klik ***start->Run*** ketik **cmd**. Setelah muncul, ketik: **SQLPLUS,** masukkan ***username*** (*default*-nya adalah **system**), dan ***password*** (jika diminta) yang dibuat saat membuat *database*, setelah itu masuk ke *database* Oracle ditandai dengan "**SQL>**". Perintah SQL bisa dijalankan setelah tulisan **"SQL>"** muncul seperti pada gambar 2-1.



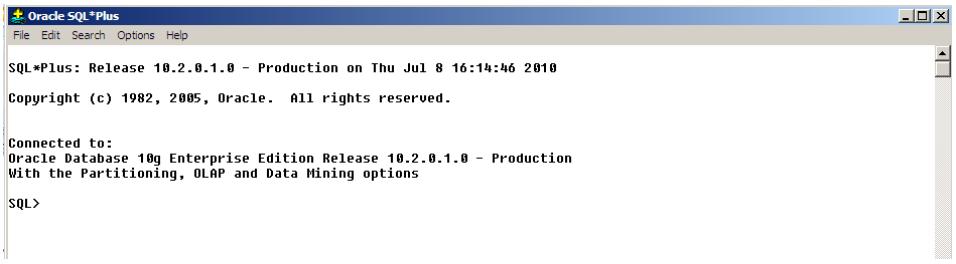
Gambar 1-26. SQL via CMD

2. **SQL*PLUS**

   Menggunakan SQL*Plus hampir sama saja seperti *Command prompt* dimana sintaks SQL yang akan di *running* harus diketikkan manual, hanya saja SQL*Plus ini *tools* bawaan dari Oracle. Untuk mengaksesnya pilih ***Start*** **→** ***All*** **Program → Oracle - OraDb10g_home1 →** ***Application Development*** **→ SQL Plus.**

Gambar 1-27. SQL Plus

Pada SQL*plus ini harus dimasukkan *username* dan *password* yang telah dibuat pada saat instalasi Oracle (atau saat membuat *database*). Untuk penggunaan PC *standalone* (yang tidak terhubung jaringan) tidak perlu memasukkan *host string*.



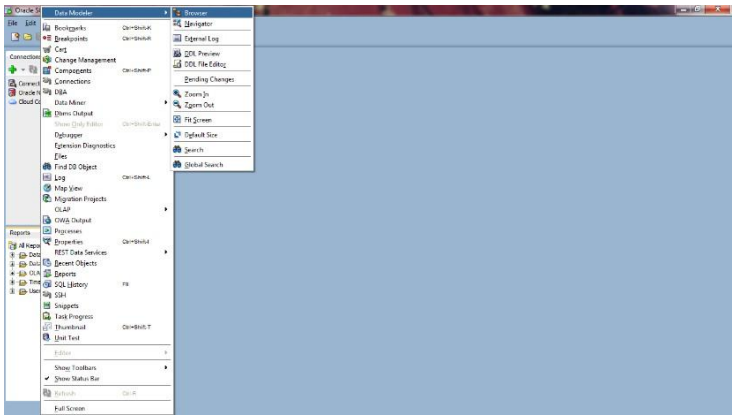
Gambar 1-28. SQL dengan logged in user

1.4 SQL Developer

1.4.1 SQL Developer pada praktikum ini digunakan untuk merancang logical model.

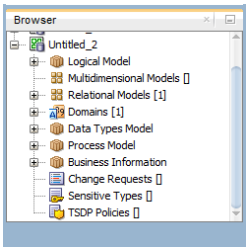
1.4.2 Berikut merupakan dasar penggunaan SQL Developer.

1. Aktifkan *Data Modeler view dengan cara:* Klik *View -> Data Modeler -> Browser*



Gambar 1-29. Data Model Browser

2. Akan muncul menu bar baru.
3. Klik kanan pada menu *Design -> New Design*

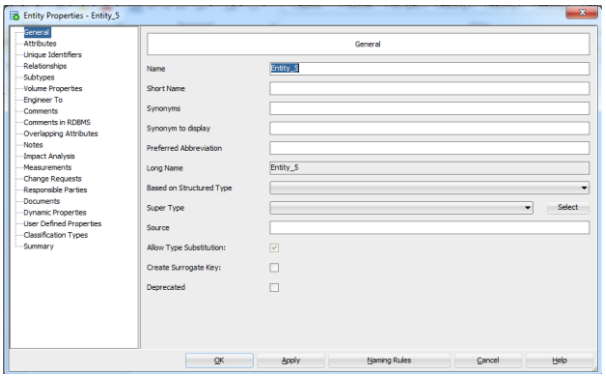


Gambar 1-30. Browser

4. Pada bagian Logical Model, klik kanan -> Show
5. Nantinya akan muncul field baru di field utama, dan muncul toolbox baru dibagian atas.
6. Untuk membuat entitas baru, klik New Entity, dan klik ke field kosong.

Gambar 1-31. New Entity

7. Akan muncul *windows* baru yang dapat mengatur entitas mulai dari nama, hingga atribut yang ada pada entitas yang dibuat.

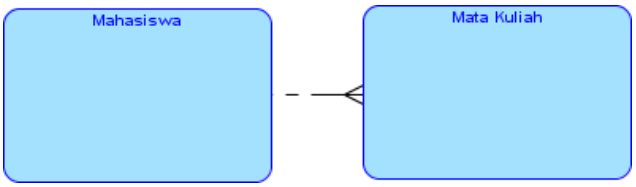


Gambar 1-32. Elemen entitas

8. Untuk membuat relasi, pilih salah satu dari relasi yang ada pada *toolbox*.

Gambar 1-33. Relasi

9. Hasil akhir sederhana.



Gambar 1-34. Model sederhana

Modul 2.2 DDL/DML, Data types, Authorization

Tujuan Praktikum
1. Mampu menjelaskan konsep dasar Oracle 11g 2. Mampu mengoperasikan DDL dalam Oracle dan mengaplikasikannya di studi kasus 3. Mengetahui dan memahami operasi-operasi *Data Manipulation Language* (DML)

2.1. Ringkasan Materi

2.1.1. Pengenalan Oracle

Oracle *Database* adalah suatu perangkat lunak *database* yang digunakan untuk menyimpan data. Biasanya perusahaan besar yang menggunakan program ini. Aplikasi ini merupakan produk yang dikembangkan oleh Oracle Corporation, pertama kali dirilis pada tahun 1979. Oracle dikembangkan oleh Larry-Ellison bersama Bob Miner, dan Ed Oates yang pada awalnya bekerja di Software Development Laboratories (SDL) pada tahun 1977. SDL kemudian berhasil membuat versi *prototype* orisinil dari Oracle software, yaitu Oracle versi 1, yang ditulis dalam bahasa C dan mempunyai SQL *interface*. Nama Oracle berasal dari *code-name* suatu proyek milik CIA dimana Larry-Ellison dulunya bekerja.

2.1.1.1 Fungsionalitas

Oracle Relational Database Management System (Oracle RDBMS) atau sederhananya Oracle', merupakan sebuah RDBMS yang mempunyai fungsi antara lain sebagai berikut:

1. *Data Storage* pada area perancangan basis data.
2. *Access Data* untuk aplikasi tertentu, dengan Teknik Optimisasi.
3. *Database Security* dan pengaturan *User Privilege* tertentu.
4. *Consistency Data* yang berfungsi untuk proteksi data, dan *locking-mechanism.*
5. *Integrity Data* yang berfungsi pada implementasi basis data tersebar.

2.1.1.2 Kelebihan

1. *Scalability*, kemampuan menangani banyak *user* yang melakukan koneksi secara simultan tanpa berkurangnya *performance* secara signifikan. Dalam dokumentasinya, Oracle menyebutkan bahwa *database* Oracle dapat melayani puluhan ribu *user* secara simultan.
2. *Reliability* yang bagus, yaitu kemampuan untuk melindungi data dari kerusakan jika terjadi kegagalan fungsi pada sistem seperti *disk failure*.
3. *Stability*, yaitu kemampuan untuk tidak *crash* karena beban yang tinggi.
4. *Availability*,yaitu kemampuan dalam penanganan *crash* atau *failure* agar *service* tetap berjalan normal.
5. *Multiplatform*, dapat digunakan pada banyak sistem operasi seperti Windows, Unix, Linux dan Solaris.
6. Mendukung data yang berukuran besar. Berdasarkan dokumentasinya, Oracle dapat menampung data sampai 512 petabyte(1 petabyte= 1024 terabyte).
7. *Security* yang cukup handal.

2.1.1.3 Kekurangan

1. Merupakan *software* DMBS yang paling mahal, paling rumit, dan paling sulit untuk dipelajari.
2. Membutuhkan spesifikasi *hardware* yang tinggi untuk dapat menjalankan *software* DMBS Oracle supaya berjalan dengan stabil.
3. Hanya diperuntukan bagi perusahaan berukuran besar, dan tidak cocok untuk perusahaan kecil maupun menengah.
4. Data yang bertambah ukurannya akan mengalami kelambatan proses, jadi harus ada *database management*.
5. Harga yang sangat mahal untuk sebuah *database* dan penggunaan Oracle sangat memakan banyak biaya, mulai dari *device* sampai diperlukannya DBA yang handal.

2.1.2 Tipe Data Pada Oracle 11g

1. *Char* (i)

String dengan panjang karakter **tetap** sebesar ukuran ***i.*** Tipe data ini mempunyai kemampuan yang hampir sama yakni menyimpan karakter, hanya saja maksimum ***size*** yang disimpan hanya sebesar **2000 *bytes***.

2. *Varchar2* (i)

String yang memiliki panjang karakter ***variable*** dengan panjang ***maximal*** sebesar ***i***. Tipe data ini memperbolehkan penyimpanan semua karakter yang dapat dimasukkan melalui keyboard. Maksimum ***size*** yang dapat disimpan sebesar **4000 *bytes*(karakter).** Tipe data ini juga memungkinkan untuk menyimpan data numerik. Biasanya ***space*** akan langsung dielimansi jika menggunakan tipe data ini.

Contoh : ”Informaticslab ” (dengan 3 spasi di akhir) dimasukkan ke dalam kolom yang didefinisikan dengan tipe data *varchar2*, maka akan menjadi “Informaticslab” (tidak ada spasi).

3. Number(p,s)

tipe data yang memiliki presisi p dan s digit dibelakang koma. Jika kita abaikan p dan s berarti dianggap sebagai tipe *number* ***floating point***. Tipe data ini menyimpan bilangan integer sampai maksimum dari digit integer tersebut. Bagian kiri menunjukkan presisi dan bagian kanan menunjukkan skala.

Contoh:

Number specification	Column Length	Decimal Digits
(4,2)	4	2
(7,4)	7	4
(15,11)	15	11

Jumlah integer yang disajikan adalah merupakan **selisih** antara kedua angka tersebut.

> **Catatan NUMBER(p,s)**
> p = presisi (total digit keseluruhan)
> s = skala (digit di belakang koma)
> Contoh: NUMBER(7,4) -> 3 digit sebelum koma, 4 digit sesudah koma.
> Jika digit desimal melebihi skala, nilai dibulatkan otomatis.

4. Long

Data karakter dengan ukuran panjang bervariasi, hingga mencapai **2 *GigaBytes***. (tipe data ini tidak dapat digunakan sebagai *key*).

5. ***Raw*(*size*)**
   Tipe data berupa *binary* data dengan ukuran maksimal **255 bytes.**

6. ***Long Raw***
   Tipe data berupa *binary* data dengan ukuran maksimal **2 GigaBytes** tidak dikonversi oleh oracle (data mentah apa adanya, spasi dihitung 1 karakter).

7. ***Date***
   Tipe data ini meyimpan **waktu** dan **tanggal** dari sebuah informasi, dengan komponen waktu yang dibulatkan ke detik terdekat. Untuk menampilkannya menjadi sebuah *text*, harus menggunakan fungsi ***to_char***.

8. ***Timestamp***
   Tipe data ini mirip dengan tipe data ***date***. Di dalam tipe data ini terdapat komponen waktu yang dapat langsung ditampilkan tanpa harus mengubahnya terlebih dahulu dengan fungsi ***to_char***.

9. ***Clob***
   Tipa data ini memperbolehkan penyimpanan objek dengan ukuran yang sangat besar sekitar **4 GB**. *Clob* mirip sekali dengan varchar2 hanya saja tipe data ini mempunyai kapasistas maksimum yang lebih besar.

10. ***Blob***
    Tipe data ini memperbolehkan penyimpanan objek *binary* yang besar dan tidak terstruktur. Contoh dari tipe data *blob* adalah **suara** dan **video**.

2.1.3 Konsep Dasar SQL

Kategori Perintah SQL

1. **DDL (Data Definition Language)** → Mengatur struktur
2. **DML (Data Manipulation Language)** → Mengatur isi data
3. **DCL (Data Control Language)** → Hak akses
4. **TCL (Transaction Control Language)** → Commit / Rollback

2.1.4 Data Definiton Language (DDL)

DDL merupakan bagian dari perintah SQL untuk membuat, memodifikasi atau menghapus struktur basis data Oracle. Suatu DDL akan dieksekusi apabila sudah diakhiri dengan ';' (titik koma).

2.1.4.1 Create

1. Create Table

Perintah Create Table digunakan untuk **membuat/mendefinisikan sebuah tabel baru**.

Sintaks :

```
CREATE TABLE nama_tabel
(Nama_atribut1  tipe_data  CONSTRAINT_pk_nama_tabel  PRIMARY KEY,
Nama_atribut2  tipe_data  [unique]  [notnull/null]  [default] <nilai default>
[check],
.............
);
```

Contoh :

```
CREATE TABLE SUPERVISOR
(
        NIP CHAR(6) NOT NULL ENABLE,
        NMSUPERVISOR  VARCHAR2(20),
        THN_MASUK NUMBER,
        CONSTRAINT SUPERVISOR_PK PRIMARY KEY (NIP) ENABLE
 );
```

Ket: Fungsi dari **CONSTRAINT** akan di bahas pada sub-bab V.

NOTE: Untuk melihat struktur sebuah *table* yang sudah dibuat dapat digunakan perintah ***DESCRIBE* nama_tabel;** atau **DESC nama_tabel;**

2. Create View

View merupakan sebuah **tabel semu / tabel lojik**, dimana datanya berasal dari satu atau lebih tabel lain sebagai tabel sumber. *View* tidak memiliki indeks, terdapat batasan dalam memperbaharui data dalam *view*. Ketika *view* diperbaharui maka perubahannya dialurkan ke tabel dasar yang mendasari referensi untuk membuat *view*. Tabel dasar merupakan tabel dalam model data relasional yang berisikan *raw* data.

Sintaks :

```
CREATE  [OR REPLACE]  [FORCE]  [NOFORCE]  VIEW  nama_view
[(alias[,alias]...)]  AS
Subquery  [with  <CHECK option>|<READ ONLY>];
```

Catatan VIEW

OR REPLACE → update definisi view kalau sudah ada
FORCE → paksa buat view walaupun ada error referensi
NOFORCE → default, gagal kalau ada error
WITH CHECK OPTION → boleh ubah data tapi tidak boleh keluar filter
WITH READ ONLY → haram hukumnya ubah data

Contoh sederhana:

```
CREATE OR REPLACE VIEW daftar_film
(id, nama_film, aktor ) AS
SELECT id_film, judul, nama_aktor
FROM film;
```

Contoh:

```
CREATE  OR  REPLACE  FORCE  VIEW  view_data_transaksi
        (Id_transaksi,  nama_member,  tgl_trans, diskon)
AS
        select  T.idtransaksi, M.nmmember, T.tgltransaksi, S.diskon
        from  transaksi T,  member M,  membership S
        where  T.idmember=M.idmember
        and S.idmembership=M.idmembership
        with  check  option;
```

3. Create Sequence (Membuat Nomor Unik Secara Otomatis)

Sequence digunakan untuk **men-*generate* angka angka unik secara otomatis**. Biasanya sequence diaplikasikan untuk membentuk *primary key* secara otomatis.

Sintaks :

```
CREATE SEQUENCE nama_sequence
        [increment by integer]
        [start with integer]
        [maxvalue integer | nomaxvalue]
        [minvalue integer | nominvalue]
        [cycle | nocycle]
        [cache integer | nocache]
        [ order | noorder]
        ;
```

Contoh :

```
CREATE SEQUENCE idMembership
        Increment by 1
        Start with 1
        Maxvalue 999999
        Minvalue 0 Nocycle
        Nocache
        Order;
```

Pada sintak pembentukan sequence terdapat beberapa kata kunci seperti berikut:

- ***INCREMENT BY***, untuk menyatakan nilai penambahan/penurunan pada *sequence*.
- ***START WITH***, untuk menyatakan nilai awal yang dihasilkan *sequence*.
- ***MAXVALUE***, untuk menyatakan nilai tertinggi sequence. *NOMAXVALUE* berarti nilai tertinggi yang dihasilkan tidak dibatasi. *MAXVALUE* dan *NOMAXVALUE* bersifat optional.
- ***MINVALUE***, untuk menyatakan nilai terendah yang dihasilkan *sequence*. *MINVALUE* berarti nilai terendah yang dihasilkan tidak dibatasi. *MINVALUE* dan *NOMINVALUE* bersifat *optional*.
- ***CYCLE***, untuk menyatakan sequence akan kembali ke nilai terendah yang ditetapkan. ***NOCYCLE*** adalah kebalikan dari *CYCLE*.
- ***CACHE***, untuk menentukan berapa banyak nilai dari sequence yang akan dialokasikan ke memori. ***NOCACHE*** berarti tidak ada nilai sequence yang dialokasikan ke memori.
- ***ORDER* atau *NOORDER*** berfungsi untuk menjamin atau tidak, nilai yang di-*generate* adalah berurut berdasarkan permintaan pada *server paralel*. ***Default*** dari klausa ini adalah ***NOORDER*.**

Untuk penggunaan sequence, dapat menggunakan :

```
Sintaks : Nama_sequence.nextval
```

Contoh saat implementasi insert:

```
INSERT INTO barang (id_barang, nama_barang)
VALUES (seq_barang.NEXTVAL, 'Laptop');
```

4. Create Synonym

Synonym digunakan untuk **membuat nama alias bagi sebuah tabel, *view* atau *sequence***. *Synonym* biasanya digunakan untuk menjaga keamanan struktur suatu tabel.

Sintaks :

```
CREATE [PUBLIC] SYNONYM nama_synonim
For nama_tabel/view/sequence;
```

Contoh :

```
CREATE SYNONYM prdk FOR produk;
```

Tujuan penggunaan *synonim* adalah:

- Untuk mengganti atau menyingkat nama yang sulit diingat dari sebuah objek.
- *User* dapat mengakses data pada *table* yang berisi informasi yang *sensitive* dan *private* melalui synonym tanpa harus mengetahui mana table aslinya sehingga kamu tidak dapat melakukan perubahan pada struktur *table*.

5. Create User

Create user digunakan untuk **mendefinisikan *user* baru beserta *password*-nya**.

Sintaks :

```
CREATE USER nama_user
IDENTIFIED BY password_user;
```

Contoh :

```
SQL> create user nunit identified by nunit123;

User created.
```

Ket: Agar *user* baru ini dapat *log on*, maka setelah di *create* lakukan GRANT CONNECT sebagai berikut:

```
GRANT CONNECT, RESOURCE TO nama_user;
```

Contoh**:**

```
SQL> GRANT CONNECT, RESOURCE TO NUNIT;

Grant succeeded.
```

6. Create Index

Index digunakan untuk **mempercepat pengaksesan data** pada suatu tabel. Index dapat diberikan pada satu atau lebih kolom.

Sintaks :

```
CREATE INDEX nama_index ON nama_tabel (kolom1 [, kolom2, ...]);
```

Contoh : **CREATE INDEX** nokartu **ON** member (nokartu)**;**

2.1.4.2 Drop

7. Drop Table

Perintah ini digunakan untuk **menghapus definisi tabel** pada ORACLE yaitu semua data dan indeks yang dimiliki tabel.

Sintaks :

```
DROP TABLE nama_table [CASCADE CONSTRAINT];
```

Contoh **:** **DROP TABLE** member CASCADE CONSTRAINT;

Ket : *Option* ***CASCADE CONSTRAINT*** akan menghapus pula *constrain referensi integrity* yang terhubung.

8. Drop View

Perintah ini digunakan untuk **menghapus *view*** tertentu yang ada di basisdata.

Sintaks :

```
DROP VIEW nama_view;
```

Contoh : **DROP VIEW** view_data_transaksi;

9. Drop Sequence

Perintah ini digunakan untuk **menghapus *sequence*** tertentu yang ada di basisdata.

Sintaks :

```
DROP SEQUENCE nama_sequence;
```

Contoh : **DROP SEQUENCE** seq_ transaksi;

10. Drop Synonym

Perintah ini digunakan untuk **menghapus *synonym*** tertentu yang ada di basisdata.

Sintaks :

```
DROP SEQUENCE nama_synonym;
```

Contoh : **DROP SEQUENCE** seq_ transaksi;

11. Drop Index

Perintah ini digunakan untuk **menghapus index** tertentu yang ada di basisdata.

Sintaks :

```
DROP INDEX nama_index;
```

Contoh : **DROP INDEX** nokartu;

12. Drop User

Perintah ini digunakan untuk **menghapus definisi *user*** tertentu.

Sintaks :

```
DROP USER nama_user;
```

Contoh : **DROP USER** nunit;

2.1.4.3 Alter

13. Alter Table

Perintah ini digunakan untuk mengubah struktur tabel dan *constraint*-nya. Pada sebuah tabel ada tiga macam alter yang dapat digunakan secara terpisah ataupun digunakan bersamaan. Ketiga perintah alter tersebut yaitu:

- ***ADD***

  Digunakan untuk **menambah kolom(atribut) atau *constraint*** pada suatu tabel.

  Sintaks :

  ```
  ALTER TABLE nama_tabel
  ADD nama_atribut_yang_hendak_ditambah  tipe_data  constraint);
  ```

  Contoh: **ALTER TABLE** supervisor **ADD** tgl_lahir date;

  Contoh 2: **ALTER TABLE** member **ADD** (
  Idmembership char(6),
  Constraint member_fk1 Foreign Key (idmembership) references membership on delete cascade) ;

- ***MODIFY***

  Digunakan untuk **mengubah definisi** (tipe data, *constraint*, atau ukuran ***size*** dari suatu tipe data) suatu kolom.

  Sintaks :

  ```
  ALTER TABLE nama_tabel
  MODIFY nama_atribut_yang_hendak_diubah  tipe_data  constraint;
  ```

  Contoh: Suatu tabel **member** mempunyai sebuah kolom **telp** yang tipe datanya awalnya **char(13),** ingin diubah menjadi tipe data **varchar2(15).** Maka sintaksnya:

  ALTER TABLE member **MODIFY** telp varchar2(15);

- ***DROP***

  Digunakan untuk **menghapus kolom(atribut) atau *constraint*** pada suatu tabel.

  Sintaks :

  ```
  ALTER TABLE nama_tabel
  DROP [COLUMN|CONSTRAINT] nama_atribut_yang_hendak_dihapus  constraint;
  ```

  Contoh: **ALTER TABLE** member
  DROP COLUMN telp; ← menghapus kolom **telp**

  Contoh 2: **ALTER TABLE** member
  DROP CONSTRAINT member_fk1; ← menghapus foreign key

- ***ENABLE/DISABLE***

  Digunakan untuk mengaktifkan/menonaktifkan *constraint*.

  ```
  ALTER TABLE nama_table ENABLE CONSTRAINT nama_constraint;
  ALTER TABLE nama_table DISABLE CONSTRAINT nama_constraint [CASCADE];
  ```

  Contoh : **ALTER TABLE** member **ENABLE CONSTRAINT** member_pk;

  Contoh **: ALTER TABLE** member **DISABLE CONSTRAINT** member_pk **CASCADE**;

  Ket: Perintah ***CASCADE*** digunakan untuk men-disable *dependent integrity constraint*.

- ***Alter View***

  Perintah ini digunakan untuk **mengkompilasi ulang** sebuah *view*.

  Sintaks:
  ```
  ALTER VIEW nama_view COMPILE;
  ```

  Contoh: **ALTER VIEW** view_data_transaksi **COMPILE**

- ***Alter Sequence***

  Perintah ini digunakan untuk **memodifikasi karaktersitik *sequence*** baik itu nilai *increment*, nilai minimum, nilai maksimum, dan lain- lain.

  Sintaks:
  ```
  ALTER SEQUENCE nama-sequence
  [INCREMENT BY integer]
  [MAXVALUE integer | NOMAXVALUE]
  [MINVALUE integer | NOMINVALUE]
  [CYCLE | NO CYCLE];
  ```

  Contoh:
  ```
  ALTER SEQUENCE seq_transaksi
        Increment by 2
        Maxvalue 1000
        Minvalue 0
        Cycle
  ```

  Ket: *Statement START WITH* tidak dapat di-alter. Untuk *statement CYCLE* harus ada inisialisasi *MAXVALUE*.

- ***Alter User***

  Perintah ini digunakan untuk **mengubah password** seorang *user*.

  Sintaks:
  ```
  ALTER USER nama-user
  IDENTIFIED BY password;
  ```

  Contoh:
  ```
  ALTER USER nunit
  IDENTIFIED BY informaticslab;
  ```

14. Rename

Perintah ini digunakan untuk mengubah nama tabel, view, sequence dan synonym.

Sintaks:
```
RENAME nama_lama TO nama_baru;
```

Contoh: **RENAME** member **TO** pelanggan;

15. Truncate

Oracle menyediakan ***TRUNCATE*** statement untuk **menghapus seluruh data dari suatu tabel** tanpa menghapus tabel yang ada **(yang dihapus isi tabelnya saja, tabel menjadi kosong)**.

Sintaks:

```
TRUNCATE TABLE [schema.] nama_table;
```

Contoh: **TRUNCATE TABLE** member;

16. Comment

User dapat **menambahkan komentar** terhadap tabel/ view. Berikut adalah perintah *comment*:

Sintaks:

```
COMMENT ON TABLE [schema.] nama_table IS 'isi komentar'
```

Contoh: **COMMENT ON TABLE** membership **IS** 'berisi jenis jenis keanggotaan yang tersedia dengan besaran diskon tertentu';

Contoh cara cek comment:

```
SELECT table_name, comments
FROM user_tab_comments
[WHERE table_name = 'MEMBERSHIP'];
```

17. Constraint

Jika diperhatikan dengan seksama, pada pendefenisian nama atribut terdapat kata kunci ***constraint***. ***Constraint*** adalah **pemaksaan aturan** yang dilakukan pada level tabel. Terdapat beberapa *constraint* yang dapat digunakan yaitu:

A. Primary key

Secara *Implisit Primary key* membentuk keunikan. **Hanya ada satu *primary key* yang diperbolehkan** untuk setiap tabel. Secara implisit *Primary key* adalah ***NOT NULL***. *Primary key* dapat dikatakan juga sebagai *constraint Unique* yang bernilai NOT NULL. Pada sebuah *Primary Key* tidak boleh diberikan *constraint Unique* tetapi diperbolehkan menggunakan *NOT NULL*.

Contoh1:

```
CREATE TABLE SUPERVISOR(
    nip char(6) not null CONSTRAINT supervisor_pk
  PRIMARY KEY,
    nmsupervisor varchar2(20),
    thn_masuk number
  );
```

Contoh2:

```
CREATE TABLE SUPERVISOR(
    nip char(6) not null enable,
    nmsupervisor varchar2(20),
    thn_masuk number,
    CONSTRAINT supervisor_pk PRIMARY KEY (nip)
  );
```

Penempatan

B. *Foreign Key*

Digunakan sebagai hubungan antar *primary* atau *unique key* lain. Biasanya sebuah ***foreign key* pada suatu tabel merupakan *primary key* tabel yang lain**. *Contraint* ini bertujuan untuk menetapkan suatu kolom atau kombinasi dari beberapa kolom menjadi *foreign key* dari sebuah tabel. *Foreign key* sering disebut sebagai *referential integrity constraint.* Kolom atau kombinasi kolom yang menjadi *Foreign Key* pada tabel anak mengacu ke kolom atau kombinasi kolom yang menjadi *Primary key* atau *Unique key* pada tabel induk.

Contoh:

```
CREATE TABLE PRODUK (
    idproduk char(6) not null,
    nmproduk varchar2(20),
    stok number,
    harga number,
    detail varchar2(50),
    spesifikasi varchar2(50),
    idkategori char(6),
    CONSTRAINT produk_pk PRIMARY KEY (idproduk) ENABLE,
    CONSTRAINT produk_fk FOREIGN KEY (idkategori)
    REFERENCES kategori (idkategori)
    ON DELETE CASCADE
);
```

Ket: ***On Delete Cascade*** untuk **membentuk keterkaitan** antara tabel yang terhubung (dalam kasus ini tabel **produk** dan **kategori**. Jika suatu baris pada tabel **kategori** dihapus, maka kolom **produk** yang terhubung juga dihapus.

C. *Null / Not Null*

Jika sebuah kolom pada *database* tidak boleh kosong, maka *constraint* ***NOT NULL*** diberikan pada kolom tersebut. Pada saat tabel diisi dengan data (misalnya melalui instruksi *INSERT*), maka kolom tersebut harus diisi. Jika kolom tidak diisi, sistem akan menolaknya.
Constraint NULL berarti bahwa kolom dapat diisi dan dapat dikosongkan.

Contoh:

```
CREATE TABLE SUPERVISOR(
    nip char(6) NOT NULL CONSTRAINT supervisor_pk primary key,
    nmsupervisor varchar2(20) NOT NULL,
    thn_masuk number
);
```

D. *Unique*

Merupakan *constraint* yang memiliki sifat yang hampir sama dengan ***primary key***, yaitu harus memiliki **nilai yang berbeda untuk setiap baris pada satu kolom yang sama**. Hanya saja, *constraint unique* mengijinkan adanya nilai ***NULL*(kosong)**.

E. Check

Constraint *Check* digunakan untuk memeriksa nilai suatu kolom apakah memenuhi suatu kriteria yang diinginkan. Kriteria-kriteria tersebut dapat berupa **Perbandingan, range, NULL, LIKE, EXISTS.**

Constraint check **tidak dapat digunakan** pada:

Acuan *pseudo-columns* seperti SYSDATE, UID, dan lain lain atau *Query* yang mengacu pada nilai pada kolom lain.

Contoh :

```
CREATE TABLE PRODUK (
    idproduk char(6) not null,
    nmproduk varchar2(20),
    stok number,
    harga number,
    detail varchar2(50),
    spesifikasi varchar2(50),
    idkategori char(6),
    CONSTRAINT produk_pk PRIMARY KEY (idproduk) ENABLE,
    CONSTRAINT produk_fk FOREIGN KEY (idkategori) REFERENCES
    kategori (idkategori) ON DELETE CASCADE
    CONSTRAINT produk_ck CHECK (stok<50)
);
```

F. Menampilkan Constraint pada Tabel

Constraint ntuk menampilkan seluruh constraint yang terdapat pada tabel, termasuk nama constraint, jenis constraint, kolom yang terlibat, serta statusnya, digunakan data dictionary pada Oracle Database yaitu USER_CONSTRAINTS dan USER_CONS_COLUMNS.

Contoh sintaks:

```
SELECT c.constraint_name,
    c.constraint_type,
    cc.column_name,
    c.status
FROM user_constraints c
LEFT JOIN user_cons_columns cc
ON c.constraint_name = cc.constraint_name
WHERE c.table_name = 'NAMA_TABLE'
ORDER BY c.constraint_name;
```

Keterangan Contraint Type:

Kode	Keterangan
P	Primary Key
R	Foreign Key (Referential)
U	Unique
C	Check Constraint

2.1.5 Data Manipulation Language (DML)

2.1.4.1. Pengenalan

Data Manipulation Language merupakan bahasa yang memungkinkan pengguna untuk mengakses dan mengubah data yang sesuai dengan model datanya selain itu DML juga merupakan konsep yang menerangkan bagaimana menambah, mengubah dan menghapus baris tabel.

2.1.4.2. Perintah-perintah DML

1. Insert

Berfungsi untuk **menambahkan** baris baru ke tabel

Sintaks:

```
INSERT INTO <nama_tabel> [(nama_kolom1, nama_kolom2, ….)]
Values (nilai1, nilai2, …);

INSERT INTO <nama_tabel> [(nama_kolom1, nama_kolom2,…)] <query>;
```

Contoh:

```
SQL> INSERT INTO SUPERVISOR VALUES('SUP006','Pambudi',2014);

1 row created.
```

Sintaks Lainnya:

```
INSERT ALL
  INTO <nama_table> VALUES (nilai, nilai, nilai)
  INTO <nama_table> VALUES (nilai, nilai, nilai)
SELECT * FROM DUAL;
```

2. Update

Berfungsi untuk **memodifikasi** nilai pada baris tabel.

Sintaks:

```
UPDATE <nama_tabel>
Set <nama_kolom1>=<nilai_ekspresi1>,
 <nama_kolom2>=<nilai_ekspresi2>,
 ….
{Where <kondisi>};
```

Contoh:

```
SQL> UPDATE supervisor SET thn_masuk=2013 WHERE nip='SUP006';

1 row updated.
```

3. Delete

Berfungsi untuk **menghapus** baris tunggal atau lebih dari satu baris berdasarkan kondisi tertentu.

Sintaks:

```
DELETE FROM <nama_tabel>
{WHERE <kondisi>} ;
```

Contoh :

```
SQL> DELETE FROM supervisor WHERE nip='SUP006';

1 row deleted.
```

Jika klausa *where* tidak digunakan maka semua data pada tabel akan terhapus. Sebagai sintaks alternatif untuk menghapus semua data pada table digunakan perintah TRUNCATE.

Sintaks:

```
TRUNCATE TABLE <table> [reusage storage];
```

4. Merge

Berfungsi untuk melakukan *update* maupun *insert* ke suatu tabel tertentu berdasarkan kondisi dari tabel lain. Dengan perintah merge ini, data akan di-*insert*-kan apabila data tersebut tidak ada di tabel tujuan dan akan di-*update*-kan apabila data tersebut telah ada di tabel tujuan.

Sintaks:

```
MERGE INTO <nama_tabel> <nama_alias_tabel>
USING (table|view|sub_query) alias
ON (join condition)
        WHEN MATCHED THEN
        UPDATE SET [(column1=column_val1), (column2=column_val2),…]
        WHEN NOT MATCHED THEN
                INSERT (column_list)
                VALUES (column_values);
```

Contoh :

```
MERGE INTO pelanggan p
USING (select * from member) m
ON m.idmember=p.idmember
WHEN MATCHED THEN
        UPDATE SET p.password=m.password
WHEN NOT MATCHED THEN
        INSERT (p.idmember) VALUES (m.idmember);
```

Penjelasan :

Query di atas akan melakukan proses ***update*** ke tabel **pelanggan** apabila idmember ada yang sama dengan idmember di tabel **member**, sedangkan apabila idmember pada tabel **member** tidak ada di tabel **pelanggan** maka akan dilakukan proses *insert* ke tabel **pelanggan** sesuai dengan nilai yang terdapat pada tabel ***member***.

2.1.6 View

2.1.6.1 Pengertian View

View merupakan sebuah **tabel semu / tabel lojik**, dimana datanya berasal dari satu atau lebih tabel lain sebagai tabel sumber. *View* tidak memiliki indeks, terdapat batasan dalam memperbaharui data dalam *view*. Ketika *view* diperbaharui maka perubahannya dialurkan ke tabel dasar yang mendasari referensi untuk membuat *view*. Tabel dasar merupakan tabel dalam model data relasional yang berisikan *raw* data.

2.1.6.2 Jenis – Jenis View

1. Dynamic view

Dynamic view merupakan sebuah tabel virtual yang dibuat secara dinamis sesuai permintaan pengguna. *Dynamic view* bukan merupakan tabel sementara, melainkan tempat menyiman definisinya dalam katalog sistem. Dan konten dari *view* diwujudkan sebagai hasil dari *query sql* yang

menggunakan *view* tersebut. Berbeda dengan *materialized view* yang dapat disimpan pada *disk* yang dapat diperbaharui sesuai interval tertentu. Pernyataan *SQL SELECT* apapun dapat digunakan untuk membuat *view*. Data yang ada disimpan pada tabel dasar, yang telah ditentukan oleh perintah *SQL CREATE TABLE*. *Dynamic view* selalu berisi nilai turunan terbaru dan karenanya lebih unggul dalam hal data *currency* daripada membuat tabel nyata sementara dari beberapa tabel dasar.

Tabel 2-1. Kelebihan dan kekurangan menggunakan *dynamic view*

Aspek positif	Aspek negatif
Menyederhanakan perintah query	Penggunaan dan waktu proses yang lama, karena kontennya dihitung setiap kali digunakan.
Membantu memberikan keamanan dan kerahasiaan data	Jika struktur tabel dasar dirubah, dan struktur tersebut direfensikan dalam *view,* maka tampilan harus dibuat ulang untuk mencegah kesalahan query.
Meningkatkan produktivitas programmer	
Berisi data tabel dasar terbaru	
Menggunakan ruangan penyimpanan yang kecil	
Memberikan *view* yang dapat disesuaikan untuk pengguna	
Membangun independensi data fisik	

2. Materialized view

Materialized view merupakan salinan atau replikasi data berdasarkan *query SQL* yang dibuat dengan cara yang sama seperti *dynamic view.* Namun *materialized view* diwujudkan sebagai tabel sehingga harus tetap diperhatikan untuk tersinkronisasi dengan tabel dasarnya yang terikat. Sama halnya dengan *dynamic view, materialized view* dapat dibangun dengan cara yang berbeda untuk berbagai tujuan. Tabelnya dapat direplikasi secara keseluruhan atau sebagian dan diperbaharui pada interval waktu yang telah ditentukan atau ketika tabel perlu untuk diakses. *Materialized view* dapat didasarkan pada *query* dari satu atau lebih tabel, sehingga dimungkinkan untuk membuat tabel ringkasan berdasarkan agregasi data.

Tabel 2-2. Kelebihan dan kekurangan menggunakan *materialized view*

Aspek positif	Aspek negatif
Menyederhanakan perintah query	Menggunakan penyimpanan yang lebih besar daripada *dynamic view*
Membantu memberikan keamanan dan kerahasiaan data	Data harus dirawat(*maintain*ed) dengan baik
Meningkatkan produktivitas programmer	Mungkin tidak berisi data tabel terbaru
Memberikan *view* yang dapat disesuaikan untuk pengguna	
Membangun independensi data fisik	
Dapat meningkatkan kinerja *query* terdistribusi, terutama jika data dalam *view* relatif statis dan tidak sering untuk diperbaharui	

3. Create View

3.1 Dynamic view

Sintaks: Sintaks dalam membuat *view* adalah sebagai berikut :

```
CREATE VIEW  nama_view AS
<query expression>
```

Query expression adalah ekpresi *query* apapun yang akan menjadi definisi dari sebuah *view*.

Contoh :

```
CREATE VIEW OutstandingStudent AS
SELECT NAME, COURSE, GRADE
FROM STUDENT
WHERE GRADE>B;
```

3.2 Materialized view

Sintaks :

```
CREATE MATERIALIZED VIEW nama_view AS
(<query expression>)
```

Query expression adalah ekpresi *query* apapun yang akan menjadi definisi dari sebuah *view*.

Contoh :

```
CREATE MATERIALIZED VIEW OutstandingStudent AS
(SELECT NAME, COURSE, GRADE
FROM STUDENT
WHERE GRADE>B);
```

2.2 Studi Kasus

Perhatikan studi kasus berikut ini!

Sebuah bioskop memiliki beberapa ruang *theater*. Setiap *theater* hanya memiliki 1 tipe kelas, 1 kapasitas dan 1 harga tertentu. Masing-masing ruang *theater* memiliki kapasitas kursi yang berbeda. Kursi –kursi pada setiap theater memiliki label yang menunjukkan posisi baris dan nomor. Seluruh kursi pada bioskop tersebut didata sebagai inventaris.

Setiap film yang diputar akan memiliki jadwal, dimana pada jadwal tersebut akan dijelaskan film yg diputar, jam penayangan, *theater* yang akan menayangkan dan lain-lain.

***Business Rule*:**

- 1 film bisa diputar pada lebih dari 1 *theater*.
- 1 theater bisa memutar lebih dari 1 film.
- Masing-masing film memiliki detail informasi terkait film tersebut

Seluruh data-data tersebut harus dapat tersimpan dalam *database* dan bisa di-*inquiry* untuk keperluan pelaporan.

Sebagai gambaran, berikut beberapa contoh data yang ingin disimpan :

- Contoh 1:
  Theater 1 memiliki 10 kursi dengan rincian: 5 baris (A,B,C,D,E). masing-masing baris diisi 2 nomor (1,2)
- Contoh 2:
  Theater 2 memiliki 20 kursi dengan rincian: 5 baris (A,B,C,D,E). Masig-masing baris diisi 4 nomor (1,2,3,4)
- Contoh 3:

Spider-Man: Far From Home(2019)

Sinopsis:

Spider-Man Far From Home menjadi film marvel pertama setelah Avengers: Endgame. Film ini menjadi sekuel lanjutan dari fil Spider Man Homecoming yang dirilis 5 Juli 2017 silam. Jon Watts didapuk menjadi sutradara dalam film ini, sedangkan skenarionya ditulis oleh Chris McKenna dan Erik Sommers. Aktor Tom Holland masih didapuk sebagai pemeran utama dalam film ini.

Dalam Spider-Man: Far From Home sebagian akan berurusan dengan bagaimana Peter menangani Decimation dan fakta bahwa Tony Stark sekarang sudah mati. Seperti yang ditunjukkan trailer terbaru, Peter akan dihadapkan dengan kematian Tony di setiap kesempatan. Dia tidak akan punya banyak waktu untuk bersedih, karena Spider-Man akan direkrut oleh Nick Fury untuk menangani beberapa urusan yang mendesak. Ternyata, Spider-Man direkrut untuk bertarung melawan makhluk kuno yang dikenal sebagai Elementals, yang meliputi Hydron, Hellfire, Magnum, dan Zephyr. Saat bertarung dengan Elementals, Spider-Man akan dibantu oleh Mysterio, karakter pemegang sihir yang dimainkan oleh Jake Gyllenhaal. Tetapi karakter baru Gyllenhaal yang misterius mungkin memiliki motif tersembunyi.
Siapakah Mysterio? Dia adalah karakter yang memiliki pengetahuan dan keterampilan yang hebat untuk menciptakan ilusi yang meyakinkan dan efek khusus. Beberapa orang berspekulasi bahwa Mysterio adalah seorang mistikus seperti Doctor Strange, meskipun dia tidak setinggi itu. Dalam kasus ini Mysterio gagal sebagai seorang mistikus tetapi masih berpegang teguh pada harapan bahwa dia bisa menjadi pahlawan. Jadi dia menggunakan ilusi, yang merupakan keahlian terbaiknya. Tentu saja, Mysterio bisa saja menggunakan gadget yang menunjang penampilannya. Intinya adalah, dia mungkin bertindak seperti pahlawan, tetapi dia akan terungkap sebagai penjahat.

MPAA : PG-13
Durasi : 2h 9min

Film tersebut akan diputar di bioskop XXI pada theater 1 dan 2. Pada theater 1, akan diputar pada tanggal 3 Juli 2019 s.d 20 Juli 2019, jam 12.30 – 14.30; 16.00 – 18.00; 19.30 – 21.30. Pada theater 2, akan diputar pada tanggal 3 Juli 2019 s.d 15 Juli 2019, jam 13.00 – 15.00; 16.00 – 18.00; 19.00 – 21.00.

2.3 Tutorial

DDL(*Data definition Language*)

1. Pastikan SQLPLUS oracle sudah berjalan dengan baik.
2. Tentukan tipe data dan ukuran untuk masing-masing tabel.

Nama Tabel:		
Atribut	Tipe data	Ukuran

Nama Tabel:		
Atribut	Tipe data	Ukuran

Nama Tabel:		
Atribut	Tipe data	Ukuran

Nama Tabel:		
Atribut	Tipe data	Ukuran

Nama Tabel:		
Atribut	Tipe data	Ukuran

Nama Tabel:		
Atribut	Tipe data	Ukuran

3. *Create* semua tabel yang ada pada skema relasi. Mulai dengan *table* yang tidak memiliki *foreign key atau* yang tidak memiliki atribut yang bergantung dengan tabel lain.

Tabel tanpa foreign key/Tabel Master	Tabel memiliki foreign key

DML (Data Manipulation language)

1. Lakukan *insert* data film sebagai berikut !

Id_film	judul	Durasi (menit)	tahun	sinopsis
F0101	Keluarga Cemara	110	2019	Fokus cerita ini masih soal, Emak serta kedua anak mereka, Euis dan Ara . Persis dengan cerita di sinetronnya, film ini mengisahkan tentang bagaimana perjalanan hidup keluarga Abah yang semula nyaman dan mapan lalu kemudian mendadak bangkrut. Abah pun mengajak keluarganya pindah ke rumah warisan yang cukup jauh dari kota. Dengan segala keterbatasan dan kekurangan yang ia memiliki, Abah berusaha menjadi kepala keluarga serta ayah yang baik untuk anak-anaknya. Untung saja, Emak tak pernah lelah mendampinginya dan berusaha menjadi sandaran di saat mereka sedih.
F0102	Habibie Ainun 3	96	2019	Habibie & Ainun 3 adalah sebuah film Indonesia tahun 2019 yang disutradarai oleh Hanung Bramantyo dan diproduksi oleh Manoj Punjabi (MD Pictures). Film ini adalah film ketiga dari seri film Habibie & Ainun. Bila "Rudy Habibie" merupakan prekuel dari kisah Habibie muda, maka "Habibie & Ainun 3" ini adalah prekuel dari kisah Ainun muda. Film ini dijadwalkan rilis pada 19 Desember 2019. Hasri Ainun Besari diperankan oleh Maudy Ayunda menggantikan Bunga Citra Lestari pada film pertama, karena film ini akan lebih menekankan pada kisah Ainun di masa muda.
F0103	Taufiq	105	2019	Kisah penjalanan hidup politisi yang juga suami Mantan Presiden RI, Megawati Soekarnoputri, Taufiq Kiemas diangkat dalam sebuah film tayang 14 Maret 2019. Film yang diangkat dari kisah nyata itu disutradarai sekaligus skenarionya ditulis oleh Ismail Basbeth. Perjalanan hidup membawanya ke Jakarta dan bertemu langsung dengan Soekarno, menjalin persahabatan dengan Guntur Soekarnoputra serta bertemu Megawati Soekarnoputri untuk pertama kalinya. Taufiq akan memahami arti dari persahabatan, keluarga, cinta dan negara ketika dia menjalani ujian hidup sebagai seorang nasionalis. Setelah itu Ia dijebloskan ke penjara bersama sahabatnya dituduh sebagai anggota Komunis. Dalam

Id_film	judul	Durasi (menit)	tahun	sinopsis
				penjara Taufiq banyak bertemu orang-orang hebat. Dari situlah perjalanan seorang lelaki yang menantang badai.
F0104	Buya Hamka	110	2019	Buya Hamka adalah film drama biografi Indonesia tentang Abdul Malik Karim Amrullah atau Hamka, seorang ulama Indonesia yang dikenal sebagai penulis, pujangga, dan politisi. Buya Hamka akan mengisahkan kehidupan Hamka sejak lahir sampai meninggal dunia. Sebelum menjadi sosok yang dikenal banyak orang, Hamka telah melewati beberapa perubahan, mulai dari kanak-kanak, remaja, hingga berkeluarga. Sebagai ulama, film ini menyoroti bagaimana cara Hamka menyampaikan dakwahnya secara santun. Selain itu, Buya Hamka menyoroti aspek humanis Hamka dan prosesnya menggapai semua pencapaian.

2. Lakukan insert data teater sebagai berikut !

Nomor_teater	Kelas	harga	kapasitas
Teater 1	Reguler	30000	50
Teater 2	Sweetbox	100000	50
Teater 3	4D	75000	50
Teater 4	Velvet	80000	50
Teater 5	3D	50000	50

3. Lakukan insert data Member sebagai berikut !

Id_member	Nama_member	No_hp	email	Tgl_lahir
MM0111	Anto	085267656789	Anto2016@gmail.com	1989-07-21
MM0112	Budi	081367589632	Budi2016@gmail.com	1985-01-03
MM0113	Ari	081267867543	Ari2016@gmail.com	1983-11-24
MM0114	Rahmi	085267935678	Rahmi2016@gmail.com	1981-09-01
MM0115	Fahmi	085767298908	Fahmi2016@gmail.com	1986-07-15
MM0116	Rusli	085643755398	Rusli2016@gmail.com	1988-02-26
MM0117	Doni	081398426789	Doni2016@gmail.com	1986-04-17
MM0118	Tati	085245289074	Tati2016@gmail.com	1985-06-22
MM0119	Dono	081287234567	Dono2016@gmail.com	1990-07-03
MM0120	Joko	081223670942	Joko2016@gmail.com	1988-07-19

4. Lakukan insert data jadwalTayang sebagai berikut !

Id_jadwalTayang	Id_film	Nomor_teater	Periode_start	Periode_end
JT001	F0101	Teater 1	01/07/2019	07/07/2019
JT002	F0101	Teater 1	01/07/2019	07/07/2019
JT003	F0102	Teater 1	01/07/2019	07/07/2019
JT004	F0101	Teater 2	01/07/2019	07/07/2019
JT005	F0103	Teater 2	01/07/2019	07/07/2019
JT006	F0102	Teater 2	01/07/2019	07/07/2019

JT007	F0104	Teater 2	01/07/2019	07/07/2019
JT008	F0102	Teater 3	01/07/2019	07/07/2019
JT009	F0101	Teater 1	01/07/2019	07/07/2019
JT010	F0102	Teater 1	01/07/2019	07/07/2019

5. Lakukan insert data kursi sebagai berikut !

No_inventori	Nomor_teater	No_kursi
IN1001	Teater 1	A1
IN1002	Teater 1	A2
IN1003	Teater 1	A3
IN1004	Teater 1	A4
IN1005	Teater 1	A5
IN1006	Teater 2	A6
IN1007	Teater 2	A7
IN1008	Teater 2	A8
IN1009	Teater 2	B1
IN1010	Teater 2	B2
IN1011	Teater 3	C1
IN1012	Teater 3	C2

6. Lakukan insert data order sebagai berikut !

Id_order	Id_member	Id_jadwalTayang	No_inventori	Tgl	Status
P10001	MM0111	JT001	IN1010	15/06/2019	pesan
P10002	MM0112	JT006	IN1009	16/06/2019	pesan
P10003	MM0113	JT008	IN1008	17/06/2019	pesan
P10004	MM0114	JT001	IN1007	18/06/2019	pesan
P10005	MM0115	JT006	IN1006	19/06/2019	pesan
P10006	MM0116	JT001	IN1001	20/06/2019	bayar
P10007	MM0117	JT008	IN1011	21/06/2019	pesan
P10008	MM0118	JT001	IN1003	22/06/2019	pesan
P10009	MM0119	JT006	IN1004	23/06/2019	pesan
P10010	MM0120	JT008	IN1005	24/06/2019	checkin

7. Lakukan Update data member sebagai berikut !
   Nomer HP member dengan id MM0113 an Ari berubah menjadi 08098933333
   Email member dengan id MM0117 an Doni menjadi doni_inod@gmail.com
   Teater 5(3D) harganya berubah menjadi 100000.

8. Lakukan penghapusan data Film dengan judul “Buya Hamka” dan teater 3 (4D) dari tabel yang sudah di buat.